NetPrint_Prototype

Formation C# / SaaS pour développeur WinDev — le portail et le premier module
Solution : C:\SAAS\NetPrint_Prototype · Visual Studio 2026 · .NET 10 · Blazor Interactive Server · charte NetPrint (HTML + app.css) · état du code au 30/10/2026

Partie 1 — Le portail NetPrint

1. L'architecture du projet, pas à pas
2. Suivre le code au débogueur dès la première ligne
3. Les paramètres `args`
4. Program.cs ligne par ligne, et ce qui se passe après `app.Run()`
5. Comment un module s'affiche dans le portail
6. Ajouter un nouveau module, étape par étape

Partie 2 — Lancement du module Consultation devis

7. Du clic à l'écran : vue d'ensemble
8. ConsultationDevis.razor ligne par ligne : comment la fenêtre s'affiche
9. Après l'affichage : événements et recherche
10. Points d'arrêt et correspondance WinDev

> **Vocabulaire du document.** Le **portail** = l'application NetPrint.Web (connexion, accueil, barre d'application, menu). Un **module** = un projet métier (ici `NetPrint.Modules.Ventes`) chargé dans le portail. Une **appli** au sens WinDev (« Consultation devis ») devient un **écran** du module.

PARTIE 1

Le portail NetPrint

Comment la solution est organisée, comment l'application démarre et comment un module s'y branche.

1. L'architecture, pas à pas

Étape 0 — L'idée générale : un « monolithe modulaire »

C'est **une seule application web** (comme un seul exécutable WinDev), découpée en **plusieurs projets** qui ont chacun un rôle précis. Règle d'or : **les dépendances vont dans un seul sens**.

```
NetPrint.AppHost          ← lanceur (développement uniquement)
   | lance
NetPrint.Web              ← LE PORTAIL, la seule application (Program.cs, connexion, menu)
   | référence
Modules/Ventes ──────►  Ventes.Contrats   (ce que les autres modules peuvent voir)
   |                          |
NetPrint.Web.Commun       ← composants d'écran partagés (dialogues, grilles…)
   |                          |
NetPrint.Noyau            ← métier partagé (sociétés, tiers…)
   |                          |
NetPrint.Socle  ◄──────────────┘   ← technique pur (session, accès DB2/PostgreSQL)
```

Point clé : le portail est **NetPrint.Web** (aidé de Web.Commun pour ses composants d'écran). Ventes n'est **pas une application séparée** comme les exécutables WinDev lancés par MenuNetprint : c'est un **module**, compilé en DLL et chargé *à l'intérieur* de NetPrint.Web. AppHost ne fait pas partie de l'application (lanceur de développement) ; Socle et Noyau sont des bibliothèques partagées.

Analogie WinDev : un `.csproj` ressemble à un composant externe ; une « référence de projet » revient à importer ce composant. En C#, le compilateur **interdit les références circulaires**, et un projet de tests vérifie en plus que personne ne « triche ».

Étape 1 — La solution et les réglages communs

- `NetPrint_Prototype.slnx` : la liste des projets, rangés en dossiers `src/`, `src/Modules/`, `tests/` (nouveau format XML de solution de VS 2026).
- `Directory.Build.props` : réglages **hérités automatiquement par tous les projets** (.NET 10, `Nullable`, `ImplicitUsings`). C'est pourquoi les `.csproj` sont si courts.

Étape 2 — Le Socle : la technique, sans aucun métier

`NetPrint.Socle` ne connaît ni le Noyau ni les modules.

Fichier	Rôle	Équivalent WinDev
`Session/SessionNetPrint.cs`	Ouvre et ferme la session ; profil, société, compte général et compte client sont stockés dans un **cookie chiffré** (claims)	Variables globales `gnSociete`, `gsProfil`
`Session/UtilisateurCourant.cs`	`IUtilisateurCourant` : **le seul moyen** pour un module de savoir qui est connecté	Lecture contrôlée de ces variables
`Session/Authentificateur.cs`	`IAuthentificateur` : vérifie profil/mot de passe par une connexion ODBC sur l'IBM i (mode démo en développement)	Ex-`SQLConnecte` de FEN_Connexion
`Session/AuthentificationNetPrint.cs`	`AjouterSocle()` : configure le cookie et enregistre les services	Code d'initialisation du projet
`Donnees/BaseDb2.cs`, `BaseNetPrint.cs`	Connexions DB2 (ODBC) et PostgreSQL, paramètres `?`, lecture des zones AS400	`HOuvreConnexion` + procédures SQL
`Modules/DescriptionModule.cs`	« Carte d'identité » d'un module : nom, URL, icône, ordre, assemblage	—
`Communs.cs`	`ResultatRecherche` (garde-fou 10 000 lignes), `RegleMetierException`	—

Deux notions C# :

- **Interface** (`IUtilisateurCourant`) : un *contrat*, une liste de méthodes sans code. La classe `UtilisateurCourant` en fournit l'implémentation. Le jour du passage à Entra ID, on change l'implémentation sans toucher aux modules.
- **`internal`** : la classe n'est visible que dans son projet ; les autres ne voient que l'interface `public`.

Étape 3 — L'injection de dépendances (notion clé)

Pas d'équivalent direct en WinDev. Au démarrage, on **enregistre** des services :

```
services.AddScoped<ISocietes, SocietesDb2>();
// « quand quelqu'un demande ISocietes, donne-lui un SocietesDb2 »
```

Une classe **demande** ce dont elle a besoin dans son constructeur, et .NET le lui fournit :

```
internal sealed class ConsultationDevisService(ConsultationDevisDb2 donnees,
IUtilisateurCourant utilisateur) : IConsultationDevis
```

Dans un écran Razor : `@inject IConsultationDevis Consultation`.

- `Singleton` : une seule instance pour toute l'application.
- `Scoped` : une instance par requête HTTP, ou **par onglet/circuit Blazor** (donc par utilisateur connecté et par onglet).

Étape 4 — Le Noyau : le métier partagé

`NetPrint.Noyau` ne contient pour l'instant que **Societes** : l'interface `ISocietes` et son implémentation `SocietesDb2` (lecture de `AA0007` en SQL paramétré). `NoyauModule.cs` expose

`AddNoyau()`. Viendront ensuite tiers, utilisateurs, référentiels : tout ce que plusieurs modules partagent.

Étape 5 — Web.Commun : la charte d'interface

Bibliothèque de **composants réutilisables** au style de la charte NetPrint. Pas de bibliothèque de composants externe : du HTML + `wwwroot/app.css` de NetPrint.Web. Équivalent des **modèles de champs / fenêtres** WinDev. Aucun métier, aucun accès base.

Fichier	Rôle
`Interface.cs`	`AddInterfaceNetPrint()` ; services `Dialogues` (alerter / confirmer, ex- `Info` / `OuiNon`) et `Notifications` (message temporaire en bas à droite)
`ZoneDialogues.razor`, `ZoneNotifications.razor`	Les zones qui affichent dialogues et notifications, placées une fois dans la mise en page
`EnteteColonne.razor`, `FiltresColonnes.cs`	Titre de colonne de grille : tri, filtre, largeur
`PreferencesGrilles.cs`	`IPreferencesGrilles` : colonnes masquées, largeurs et tri mémorisés par utilisateur et société, côté serveur
`GrilleJs.cs` + `wwwroot/grille.js`	Pont C# ↔ JavaScript pour le redimensionnement des colonnes à la souris

Étape 6 — Un module (structure type)

```
NetPrint.Modules.Ventes/
   VentesModule.cs   ← AddModuleVentes() + MapModuleVentes()
   Ecrans/           ← pages Blazor (.razor) : affichage uniquement
   Services/         ← règles métier
   Donnees/          ← SQL
NetPrint.Modules.Ventes.Contrats/   ← interfaces + objets échangés, AUCUN code
```

- `AddModuleVentes()` déclare la `DescriptionModule` (« Consultation devis », `/ventes/devis`) et enregistre les services (`IConsultationDevis`, `ConsultationDevisDb2`, `EtatConsultationDevis`).
- `MapModuleVentes()` déclare l'API `/api/ventes/...`.
- Chemin toujours **Écran → Service → Données** ; un écran ne touche jamais à la base.
- **Contrats** : Production référencera `Ventes.Contrats` (et appellera `IConsultationDevis`), jamais `Ventes` lui-même.
- `_Imports.razor` contient `@attribute [Authorize]` : tous les écrans exigent une session.

Comment un module devient visible et comment en ajouter un : chapitres 5 et 6. Le détail de l'écran Consultation devis : partie 2.

Étape 7 — NetPrint.Web : là où tout s'assemble

- `Program.cs` : le point d'entrée (détaillé au chapitre 4).
- `Composants/App.razor` : page HTML racine ; rendu statique pour la connexion, interactif pour le reste.
- `Routes.razor` : le routeur ; `AuthorizeRouteView` renvoie vers `RedirectionConnexion` si personne n'est connecté.
- `Layout/MainLayout.razor` : la **barre d'application** (logo, nom du module courant, société) et le **menu utilisateur** (bouton avec les initiales) qui liste Accueil, les modules, Paramètres et Déconnexion. Contient aussi `ZoneDialogues` et `ZoneNotifications`.

- `Pages/Accueil.razor` : une tuile par module. `Pages/Parametres.razor` : changer de société, de compte général ou de compte client sans se reconnecter.

Étape 8 — Le parcours d'une connexion

1. Le navigateur appelle `/`. Personne n'est connecté : `RedirectionConnexion` envoie vers `/connexion?retour=/`.
2. `Connexion.razor` est un formulaire HTML classique, rendu en statique.
3. Il est posté sur `EndpointsCompte.ConnexionAsync`, qui contrôle la saisie, appelle `IAuthentificateur.VerifierAsync`, puis `ISocietes.LireNomAsync`, puis `SessionNetPrint.OuvrirAsync` (pose du cookie).
4. Retour sur `/` : `MainLayout` lit `IUtilisateurCourant` et affiche la société ; l'accueil affiche les tuiles.

> Pourquoi un POST classique plutôt que Blazor ? Un cookie ne peut être posé que par une vraie réponse HTTP. Même principe pour la page Paramètres (`/compte/parametres`).

Étape 9 — Les projets autour

- **NetPrint.AppHost** (Aspire) : lance NetPrint.Web et ouvre un tableau de bord (journaux, erreurs). Développement uniquement.
- **NetPrint.Reprise** : outil console de copie DB2 → PostgreSQL, indépendant.
- **db/migrations** : futurs scripts SQL numérotés appliqués par DbUp (vide pour l'instant).
- **tests/NetPrint.Architecture.Tests** : vérifie automatiquement les règles (Socle indépendant, modules via Contrats, écrans sans accès base). À lancer avant chaque commit.

Remarques du formateur

1. **Versions flottantes** dans les projets de tests (`xunit 2.*`, `Microsoft.NET.Test.Sdk 17.*`) : deux postes peuvent compiler avec des versions différentes ; fixer des versions précises.
2. Le README de **NetPrint.Reprise** vient des essais (autre chemin, .NET 9).
3. Le **contrôle d'accès à la société** (PRT000P1 / STE) n'est pas encore fait à la connexion.
4. Le code `0xffffffff` dans la fenêtre Sortie à l'arrêt du débogage est normal : ce n'est pas un plantage.

2. Suivre le code dès la première ligne

Tout dépend du **projet de démarrage** (celui affiché en gras dans l'Explorateur de solutions).

Le plus simple : F10, sans point d'arrêt

Quand rien ne tourne : **Déboguer > Pas à pas principal (F10)**. Visual Studio démarre le projet et s'arrête sur sa première ligne. Ensuite **F10** avance d'une ligne, **F11** entre dans la méthode appelée.

Suivre l'application elle-même (recommandé)

1. Clic droit sur **NetPrint.Web** > **Définir comme projet de démarrage**.
2. Point d'arrêt (F9) dans `src/NetPrint.Web/Program.cs`, **ligne 11** :

```
CultureInfo.DefaultThreadCurrentCulture = CultureInfo.DefaultThreadCurrentUICulture = new
CultureInfo("fr-FR");
```

Les `using` au-dessus ne s'exécutent pas (ce sont des déclarations). Il n'y a pas de `Main()` : c'est le style « instructions de niveau supérieur », le compilateur génère le `Main` et la première ligne du fichier *est* le point d'entrée.

Si AppHost reste le projet de démarrage

La toute première ligne exécutée est dans `src/NetPrint.AppHost/AppHost.cs` :

```
var builder = DistributedApplication.CreateBuilder(args);
```

Ce code ne fait que lancer NetPrint.Web dans un **autre processus**. Visual Studio s'attache aussi au processus Web : le point d'arrêt de la ligne 11 de son `Program.cs` sera bien atteint.

À savoir ensuite

- `Program.cs` s'exécute **une seule fois**, jusqu'à `app.Run()` ; ensuite l'application attend les requêtes (chapitre 4, section 11).
- Pour suivre la connexion : point d'arrêt sur la première ligne de `ConnexionAsync` dans `src/NetPrint.Web/Compte/EndpointsCompte.cs` :

  ```
  var profil = (f.Profil ?? "").Trim().ToUpperInvariant();
  ```

- Pour suivre l'ouverture du module : voir partie 2, chapitre 10.
- F11 n'entre que dans votre code, pas dans celui de .NET : option **Uniquement mon code** (Outils > Options > Débogage).

3. Les paramètres `args`

`args` = les **paramètres de la ligne de commande** passés au programme. Équivalent de `LigneCommande()` en WinDev.

D'où vient cette variable ?

Dans un fichier sans `Main()`, le compilateur génère en coulisse :

```
static void Main(string[] args)   // ← args est déclaré ici, invisiblement
{
    var builder = DistributedApplication.CreateBuilder(args);
    ...
}
```

C'est un `string[]` : un élément par mot passé derrière le nom de l'exe.

À quoi il sert ici ?

Transmis à `CreateBuilder(args)`, les paramètres deviennent **de la configuration**, au même titre que `appsettings.json`, les user secrets et les variables d'environnement. Une valeur passée en ligne de commande **écrase** les autres. Exemple du README de NetPrint.Reprise :

```
NetPrint.Reprise.exe --ConnectionStrings:Db2="<chaîne ODBC>" --cible="Host=localhost;..."
```

Le `:` descend dans l'arborescence JSON (`"ConnectionStrings": { "Db2": ... }`). De même, `NetPrint.Web.exe --Db2:Bibliotheque=NP11T` pointerait vers une autre bibliothèque sans modifier `appsettings.json`.

Et avec F5 ?

`args` est vide, sauf si on renseigne des arguments :

- clic droit sur le projet > **Propriétés > Déboguer > Ouvrir l'interface utilisateur des profils de lancement** > « Arguments de ligne de commande » ;
- ou le champ `"commandLineArgs"` de `Properties/launchSettings.json`.

Au débogueur, survolez `args` : vous verrez `string[0]`, un tableau vide.

4. Program.cs ligne par ligne

> `Program.cs` ne *lance* pas NetPrint.Web, il **est** NetPrint.Web : son point d'entrée.
> `F5 → AppHost.cs ──démarre le processus──► NetPrint.Web.exe → Program.cs (1re ligne)`

La structure en deux phases

1. **Avant `Build()`, les `Add...`** : on **déclare** les services dont l'application aura besoin.
2. **Après `Build()`, les `Use...` et `Map...`** : par quoi passe chaque requête HTTP (`Use`) et quelle URL mène où (`Map`).
3. `app.Run()` démarre le serveur web et attend les requêtes.

Astuce : cliquez sur une fonction et faites **F12** (« Atteindre la définition ») pour savoir d'où elle vient.

1. La culture (ligne 11)

```
CultureInfo.DefaultThreadCurrentCulture = CultureInfo.DefaultThreadCurrentUICulture = new
CultureInfo("fr-FR");
```

Origine : .NET (`System.Globalization`). **Utilité :** dates en `30/10/2026`, montants en `1 234,56`, quel que soit le serveur.

2. Le constructeur d'application (ligne 13)

```
var builder = WebApplication.CreateBuilder(args);
```

Origine : ASP.NET Core (inclus par `Sdk="Microsoft.NET.Sdk.Web"`). **Utilité :** prépare

- `builder.Configuration` : `appsettings.json` → `appsettings.Development.json` → user secrets → variables d'environnement → `args` (chacun écrase le précédent) ;
- `builder.Environment` : « Development » ou « Production » (`ASPNETCORE_ENVIRONMENT` dans `launchSettings.json`) ;
- `builder.Services` : la liste des services à remplir avec les `Add...`.

3. Blazor et l'interface NetPrint (lignes 16 à 18)

```
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
builder.Services.AddInterfaceNetPrint();   // dialogues, notifications
```

Appel	Origine	Utilité
`AddRazorComponents()`	ASP.NET Core (Blazor)	Permet d'afficher des pages `.razor`
`.AddInteractiveServerComponents()`	ASP.NET Core (Blazor)	L'écran **vit sur le serveur** ; un clic est envoyé au serveur via SignalR, qui renvoie ce qui change
`AddInterfaceNetPrint()`	`NetPrint.Web.Commun/Interface.cs`	Enregistre `Dialogues`, `Notifications` et `IPreferencesGrilles`, utilisés par tous les écrans

4. Vos couches (lignes 21 à 25)

```
builder.AjouterSocle();
builder.Services.AddNoyau();
builder.Services.AddModuleVentes();
```

Appel	Origine	Utilité
`AjouterSocle()`	`NetPrint.Socle/Session/AuthentificationNetPrint.cs`	Cookie de session, autorisation, `IUtilisateurCourant`, `IAuthentificateur`
`AddNoyau()`	`NetPrint.Noyau/NoyauModule.cs`	`ISocietes` → `SocietesDb2`
`AddModuleVentes()`	`Modules/NetPrint.Modules.Ventes/VentesModule.cs`	`DescriptionModule` « Consultation devis » et ses services

Méthodes d'extension : `IServiceCollection` est un type Microsoft ; il « gagne » `AddNoyau()` grâce au mot-clé `this` :

```
public static IServiceCollection AddNoyau(this IServiceCollection services)
```

On peut ainsi écrire `services.AddNoyau()` au lieu de `NoyauModule.AddNoyau(services)`. `AjouterSocle()` s'applique au `builder` car il a besoin de `builder.Configuration` (dossier des clés) et de `builder.Environment` (HTTPS ou non).

5. La construction (ligne 27)

```
var app = builder.Build();
```

Origine : ASP.NET Core. **Utilité :** fige les services et crée l'application ; plus d'`Add...` possible ensuite.

6. Le pipeline (lignes 29 à 39, l'ordre compte)

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/erreur", createScopeForErrors: true);
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.UseAntiforgery();
```

Chaque `Use...` (ASP.NET Core) est un **filtre** que chaque requête traverse dans l'ordre.

Filtre	Utilité
`UseExceptionHandler("/erreur")`	En production, erreur non gérée → page `Erreur.razor` ; en développement, page détaillée
`UseHsts()`	Le navigateur n'utilise plus que HTTPS pour ce site
`UseHttpsRedirection()`	Redirige `http://` vers `https://`
`UseAuthentication()`	**Lit le cookie** `NetPrint.Session` et remplit `HttpContext.User`
`UseAuthorization()`	Vérifie `[Authorize]` / `RequireAuthorization()` : redirection vers `/connexion` (ou 401 pour l'API)
`UseAntiforgery()`	Contrôle le jeton anti-falsification des formulaires postés

> Si `UseAuthorization` passait avant `UseAuthentication`, on vérifierait les droits d'un utilisateur pas encore identifié.

7. Les routes (lignes 41 à 47)

```
app.MapStaticAssets();
app.MapEndpointsCompte();
app.MapModuleVentes();
```

Appel	Origine	Utilité
`MapStaticAssets()`	ASP.NET Core (.NET 9+)	Sert `app.css`, `netprint.png`, `grille.js`… ; compression et cache
`MapEndpointsCompte()`	`NetPrint.Web/Compte/EndpointsCompte.cs`	`POST /compte/connexion`, `/compte/deconnexion`, `/compte/parametres`
`MapModuleVentes()`	`VentesModule.cs`	API `GET /api/ventes/devis/...`

8. Retrouver les écrans des modules (lignes 50-51)

```
var assemblagesModules = app.Services.GetServices<DescriptionModule>()
    .Select(m => m.Assemblage).Distinct().ToArray();
```

Origine : `GetServices` (ASP.NET Core) ; `Select`, `Distinct`, `ToArray` (LINQ). **Utilité :** récupérer les DLL de tous les modules enregistrés.

En WinDev :

```
POUR TOUT m DE ListeDesModules
    SI PAS DéjàDansTableau(m.Assemblage) ALORS Ajoute(tab, m.Assemblage)
FIN
```

`m => m.Assemblage` est une **lambda** : mini-fonction anonyme « pour chaque m, prends m.Assemblage ».

9. Brancher Blazor (lignes 53 à 55)

```
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode()
    .AddAdditionalAssemblies(assemblagesModules);
```

- `<App>` : page racine `Composants/App.razor` ; toutes les pages `@page "/..."` de NetPrint.Web deviennent des URL.
- `.AddInteractiveServerRenderMode()` : ouvre le canal SignalR (`/_blazor`).
- `.AddAdditionalAssemblies(...)` : cherche aussi les pages **dans les DLL des modules** ; `/ventes/devis` devient accessible sans que NetPrint.Web connaisse chaque écran.

C'est grâce à cette ligne qu'ajouter un module ne demande que deux lignes dans `Program.cs`.

10. Le démarrage (ligne 57)

```
app.Run();
```

Origine : ASP.NET Core. **Utilité :** démarre le serveur web intégré (**Kestrel**) sur `http://localhost:5100` et **bloque** jusqu'à l'arrêt. Tout le code suivant s'exécute en réponse à des requêtes.

Exercice : point d'arrêt ligne 11, avancez avec F10. Sur `builder.AjouterSocle();`, faites F11 pour entrer dans le Socle. Sur `app.Run();`, un F10 fait « disparaître » le débogueur : c'est normal, l'application attend les requêtes du navigateur.

11. Après `app.Run()` : un serveur qui attend, pas un écran

`app.Run()` démarre un **serveur web** qui écoute sur `http://localhost:5100`. À ce moment-là, **rien n'est affiché nulle part** : le programme attend qu'un navigateur lui envoie une requête.

Si une page s'ouvre quand vous faites F5, c'est **Visual Studio** qui lance le navigateur (option `launchBrowser` dans `launchSettings.json`), pas `Program.cs`.

La suite, une fois le navigateur ouvert (détail : chapitre 1, étape 8) :

1. Le navigateur demande `/`.
2. Personne n'est connecté : redirection vers `/connexion`.
3. Après la connexion, la **page d'accueil** (le portail) s'affiche avec ses tuiles et la barre d'application.
4. **Là**, on attend un clic sur un module (suite : partie 2).

Enchaînement complet : `app.Run()` → **attente d'un navigateur** → connexion → portail → **attente d'un clic**.

WinDev	NetPrint (Blazor Server)
Un exe = **un utilisateur**, une fenêtre qui attend ses clics	Un serveur = **tous les utilisateurs en même temps**
La fenêtre attend un événement	Le serveur attend des requêtes ; **chaque navigateur ouvert** a sa propre session (son « circuit ») sur le serveur

> C'est plus proche d'un **serveur WebDev** que d'une application WinDev. Si dix personnes se connectent, c'est le même `app.Run()` qui les sert toutes, chacune avec sa société et son profil : d'où l'importance de `IUtilisateurCourant` (chapitre 1, étape 2).

> **Exercice :** point d'arrêt dans `OnInitializedAsync` de `MainLayout.razor` (ligne 71), puis connectez-vous. Le débogueur s'y arrête *après* `app.Run()`, au moment où votre navigateur affiche le portail.

5. Comment un module s'affiche dans le portail

Trois mécanismes distincts, souvent confondus au début.

a) La référence de projet : rendre la DLL disponible

Dans `NetPrint.Web.csproj` :

```
<ProjectReference Include="..\Modules\NetPrint.Modules.Ventes\NetPrint.Modules.Ventes.csproj"
/>
```

C'est l'équivalent d'« importer le composant externe » en WinDev. Sans elle, rien ne marche : la fonction `AddModuleVentes()` est introuvable.

b) Le `using` : un raccourci d'écriture

Ligne 2 de `Program.cs` :

```
using NetPrint.Modules.Ventes;
```

Il évite d'écrire le nom complet `NetPrint.Modules.Ventes.VentesModule.AddModuleVentes(builder.Services)`. Pour une méthode d'extension (avec `this`), il est en plus nécessaire pour que la syntaxe courte `builder.Services.AddModuleVentes()` soit proposée.

> Ce n'est **pas** `using NetPrint.Socle...` qui donne accès à `AddModuleVentes()`. `using NetPrint.Socle.Modules;` sert à utiliser le **type** `DescriptionModule` plus bas dans `GetServices<DescriptionModule>()`.

c) `AddModuleVentes()` : déclarer, pas afficher

Comme toutes les lignes `Add...` (phase 1 de `Program.cs`), elle ne fait qu'**enregistrer** :

- la **`DescriptionModule`** : nom « Consultation devis », URL `/ventes/devis`, icône 📄, ordre 2 ;
- les **services** du module (`IConsultationDevis`, `ConsultationDevisDb2`, `EtatConsultationDevis`).

L'affichage arrive **plus tard**, quand d'autres lisent ces informations :

Qui	Quand	Ce qu'il fait de la DescriptionModule
`Program.cs` → `AddAdditionalAssemblies`	Au démarrage	Rend les écrans de la DLL Ventes accessibles par URL
`MainLayout.razor`	À chaque affichage	Crée l'entrée du **menu utilisateur** et affiche le **nom du module courant** dans la barre
`Accueil.razor`	Sur la page `/`	Crée la **tuile**
L'utilisateur	Clic sur la tuile ou le menu	L'écran `/ventes/devis` s'affiche (partie 2)

> **En résumé :** NetPrint.Web est la seule application. Elle **référence** le projet Ventes (`.csproj`), **importe son espace de noms** (`using`) et **enregistre** le module au démarrage (`AddModuleVentes()`). Le menu, l'accueil et le routeur **lisent ensuite** cet enregistrement.
>
> **Expérience :** mettez en commentaire `builder.Services.AddModuleVentes();` et `app.MapModuleVentes();` . Ça compile toujours, mais « Consultation devis » disparaît du menu et de l'accueil, et `/ventes/devis` renvoie une page introuvable.

6. Ajouter un nouveau module

Exemple : Production. Convention de nom : `AddModule` + nom du module, soit `AddModuleProduction()` . Écrire cette ligne dans `Program.cs` est nécessaire, mais ce n'est qu'une des cinq étapes.

Étape 1 — Créer les deux projets

Dans `src/Modules/` :

- **`NetPrint.Modules.Production`** : copier le `.csproj` de Ventes ; dossiers `Ecrans/` , `Services/` , `Donnees/` , et un `_Imports.razor` avec `@attribute [Authorize]` .
- **`NetPrint.Modules.Production.Contrats`** : ce que les autres modules auront le droit de voir.

Étape 2 — Écrire ProductionModule.cs (là où naît la fonction)

```
namespace NetPrint.Modules.Production;

public static class ProductionModule
{
    public static IServiceCollection AddModuleProduction(this IServiceCollection services)
    {
        services.AddSingleton(new DescriptionModule(
            Code: "PRODUCTION",
            Nom: "Production",
            Url: "/production/...",       // écran d'arrivée
            Icone: "🏭",
            Ordre: 3,                     // place dans le menu
            Assemblage: typeof(ProductionModule).Assembly,
            Description: "Dossiers de fabrication, OF..."));

        // services.AddScoped<IXxxService, XxxService>();
        return services;
    }

    public static IEndpointRouteBuilder MapModuleProduction(this IEndpointRouteBuilder app)
    {
        var api = app.MapGroup("/api/production").RequireAuthorization();
        // api.MapGet(...)
        return app;
    }
}
```

Étape 3 — Brancher le module dans NetPrint.Web

a) Référence de projet dans `NetPrint.Web.csproj` (sans elle, la fonction est introuvable) :

```
<ProjectReference
Include="..\Modules\NetPrint.Modules.Production\NetPrint.Modules.Production.csproj" />
```

b) Trois lignes dans `Program.cs` :

```
using NetPrint.Modules.Production;          // en haut
...
builder.Services.AddModuleProduction();     // sous AddModuleVentes()
...
app.MapModuleProduction();                  // sous MapModuleVentes()
```

> N'oubliez pas le **`Map...`** : sans lui, les écrans marchent mais l'API `/api/production/...` n'existe pas.

Étape 4 — Mettre à jour les tests d'architecture

Dans `tests/NetPrint.Architecture.Tests/ReglesArchitecture.cs` :

```
private static readonly Assembly[] Modules =
[
    typeof(VentesModule).Assembly,
    typeof(ProductionModule).Assembly,   // ← ajout
];
```

Ajouter aussi la référence au projet dans le `.csproj` des tests. Sinon les règles ne sont **pas vérifiées** pour le nouveau module.

Étape 5 — Ajouter les projets à la solution

Clic droit sur le dossier `Modules` > **Ajouter > Projet existant** (met à jour `NetPrint_Prototype.slnx`).

Ce que vous n'avez PAS à faire

Pas besoin de toucher au menu, à l'accueil ni au routeur : `MainLayout`, `Accueil` et `AddAdditionalAssemblies` lisent la `DescriptionModule` et s'adaptent tout seuls (chapitre 5).

> **Si Production a besoin d'un devis :** dans le `.csproj` de Production, référencez `NetPrint.Modules.Ventes.Contrats`, jamais `NetPrint.Modules.Ventes`. Le test d'architecture vous le rappellera en cas d'erreur.

Récapitulatif

#	Où	Quoi
1	`src/Modules/`	Projets `NetPrint.Modules.Xxx` et `NetPrint.Modules.Xxx.Contrats`
2	`XxxModule.cs`	`AddModuleXxx()` (DescriptionModule + services) et `MapModuleXxx()` (API)
3	`NetPrint.Web.csproj` + `Program.cs`	Référence de projet, `using`, `AddModuleXxx()`, `MapModuleXxx()`
4	`ReglesArchitecture.cs` + `.csproj` des tests	Ajouter l'assemblage dans `Modules`
5	Solution	Ajouter les projets à `NetPrint_Prototype.slnx`

PARTIE 2

Lancement du module Consultation devis

Ce qui se passe entre le clic sur « Consultation devis » dans le portail et l'écran prêt à l'emploi, puis pendant son utilisation.

7. Du clic à l'écran : vue d'ensemble

Fichier principal : `src/Modules/NetPrint.Modules.Ventes/Ecrans/Devis/ConsultationDevis.razor`. Dans Visual Studio : **Explorateur de solutions > src > Modules > NetPrint.Modules.Ventes > Ecrans > Devis**. C'est la reprise de **FEN_ConsultationDevis** du WinDev.

```
Clic (tuile ou menu)
  ↳ Routeur (Routes.razor) : quelle page porte @page "/ventes/devis" ?  + session ouverte ?
       ↳ MainLayout : la barre affiche « Consultation devis »
            ↳ ConsultationDevis.razor
                 1. @inject     : .NET fournit les services
                 2. OnInitializedAsync()     ← premier code de l'écran
                 3. rendu HTML               ← la fenêtre s'affiche
                 4. OnAfterRenderAsync(true) ← après le premier affichage
                 5. attente des actions de l'utilisateur (Rechercher, double-clic…)
```

Étape 1 — Le clic

Deux points d'entrée possibles :

- **Une tuile de l'accueil**, `NetPrint.Web/Composants/Pages/Accueil.razor` ligne 15. C'est un simple lien `<a href="/ventes/devis">`, sans code C# : Blazor intercepte le lien et navigue sans recharger la page. **Pas de point d'arrêt possible.**
- **Le menu utilisateur**, `NetPrint.Web/Composants/Layout/MainLayout.razor` ligne 35 (`@onclick="() => Aller(m.Url)"`), qui appelle les lignes 94 à 98 :

```
private void Aller(string url)
{
    menuOuvert = false;
    Navigation.NavigateTo(url);     // ← équivalent d'Ouvre() en WinDev
}
```

Étape 2 — Le routeur cherche l'écran

`Routes.razor` cherche quel composant porte `@page "/ventes/devis"`. Il le trouve dans la DLL Ventes, grâce à `AddAdditionalAssemblies` (chapitre 4, section 9). `AuthorizeRouteView` vérifie au passage que la session est ouverte (le `[Authorize]` du `_Imports.razor` du module) ; sinon, retour à la connexion. Code .NET : pas de point d'arrêt ici.

Étape 3 — La barre d'application se met à jour

`MainLayout.razor` est la « fenêtre mère » : il reste affiché et place l'écran du module à l'endroit de `@Body` (ligne 51). Comme l'adresse a changé, l'événement `LocationChanged` appelle `Changement` (ligne 76), qui appelle `TrouverModule()` (ligne 83) : le premier segment de l'URL (`/ventes`) désigne le module, et son nom s'affiche dans la barre (ligne 15).

Étapes 4 à 7 — L'écran lui-même

Détaillées ligne par ligne aux chapitres 8 et 9.

8. ConsultationDevis.razor ligne par ligne

Un fichier `.razor` a deux parties, comme une fenêtre WinDev a sa **description** (les champs) et son **code** :

- **lignes 1 à 362** : les directives (`@page`, `@inject`) puis le **HTML** de la fenêtre, mêlé de C# (`@if`, `@foreach`, `@bind`, `@onclick`) ;
- **lignes 364 à 681** : le bloc `@code { }`, c'est-à-dire les variables et les procédures de la fenêtre.

L'ordre d'**exécution** n'est pas l'ordre du fichier : .NET crée d'abord l'objet (variables du `@code`), appelle `OnInitializedAsync`, *puis* produit le HTML. On suit ici l'ordre d'exécution.

8.1 Les directives (lignes 1 à 16)

Ligne	Code	Rôle
1	`@page "/ventes/devis"`	L'adresse de l'écran : c'est ce que le routeur cherche à l'étape 2
2-7	`@* ... *@`	Commentaire : écran repris de FEN_ConsultationDevis, fonctions pas encore reprises
8	`@inject IConsultationDevis Consultation`	Le service métier (contrat dans `Ventes.Contrats`, implémentation `ConsultationDevisService`, enregistré par `AddModuleVentes()`)
9	`@inject EtatConsultationDevis Etat`	La « mémoire » de l'écran : critères, lignes, tri, filtres, sélection. **Scoped** : une par circuit, elle survit quand on part sur le détail d'un devis et qu'on revient
10	`@inject ...IUtilisateurCourant`	Qui est connecté (Socle)
11	`@inject NavigationManager Navigation`	Pour ouvrir un autre écran (`NavigateTo`) — fourni par Blazor
12-13	`Dialogues`, `Notifications`	Messages à l'utilisateur (Web.Commun, enregistrés par `AddInterfaceNetPrint()`)
14	`IPreferencesGrilles`	Colonnes masquées, largeurs, tri mémorisés (Web.Commun)
15	`IJSRuntime JS`	Pour appeler du JavaScript (redimensionnement des colonnes) — fourni par Blazor
16	`@implements IAsyncDisposable`	L'écran aura une méthode de nettoyage `DisposeAsync` (ligne 476), appelée à sa fermeture

> **WinDev :** les `@inject` remplacent les variables globales et les appels directs aux procédures d'un autre projet. L'écran **reçoit** ce dont il a besoin au lieu d'aller le chercher.

8.2 Les variables de la fenêtre (lignes 364 à 426)

Créées en premier, avant tout autre code de l'écran.

Ligne	Code	Rôle
368-369	`record Colonne(...)`	Description d'une colonne de grille : clé, libellé, largeur, type de filtre, valeur de tri
373-405	`Colonnes = [ ... ]`	Les 30 colonnes de TABLE_Devis, dans l'ordre du WinDev. `l => l.Quantite` est une lambda (chapitre 4, section 8) qui dit où lire la valeur
407	`NomGrille`	Nom sous lequel les préférences de la grille sont mémorisées
411	`C => Etat.Criteres`	Raccourci : `C.Numero` au lieu de `Etat.Criteres.Numero`
412-416	`contexte`, `lignesAffichees`, `chargement`, `menu`, `menuColonnes`	Variables locales de l'écran (non conservées au retour)
419-422	`colonnesVisibles`, `largeurTable`, `tableGrille`, `grille`	Affichage de la grille ; `tableGrille` désignera la balise `<table>` (ligne 263)
424, 426	`Prefs`, `Selection`	Raccourcis vers `Etat`

Propriété avec `=>` : `private CriteresDevis C => Etat.Criteres;` n'est pas une copie. C'est un alias recalculé à chaque lecture.

8.3 OnInitializedAsync (lignes 428 à 460) : le premier code de l'écran

Appelée automatiquement par Blazor **avant** le premier affichage. Équivalent du code « Fin d'initialisation » d'une fenêtre WinDev.

```
protected override async Task OnInitializedAsync()
{
    if (Etat.Preferences is null)                                    // 431
    {
        var p = await PreferencesGrilles.LireAsync(NomGrille);       // 433
        p.Masquees.RemoveAll(cle => ...);                            // 434
        Etat.Preferences = p;                                        // 435
        if (Colonnes.Any(c => c.Cle == p.TriCle)) { ...tri... }      // 436-440
    }
    CalculerColonnes();                                              // 442

    try
    {
        contexte = await Consultation.LireContexteAsync();           // 446
        if (contexte.RepresentantImpose is { } rep) C.Representant = rep;   // 447
        if (C.CompteGeneral is null)                                 // 448
        {
            var u = await UtilisateurCourant.LireAsync();            // 450
            C.CompteGeneral = u.CompteGeneral;                       // 451
            if (u.CompteClient > 0) C.CompteClient = u.CompteClient;  // 452
        }
    }
    catch (Exception ex)
    {
        Notifications.Erreur($"Lecture des paramètres impossible : {ex.Message}");  // 457
    }
    Actualiser();                                                    // 459
}
```

Ligne	Ce qui se passe
431	Les préférences ne sont lues **qu'une fois par session** : au retour du détail d'un devis, `Etat.Preferences` est déjà rempli
433	Lecture du fichier `App_Data/preferences/<profil>-<société>.json` côté serveur (jamais dans le navigateur : règle de la charte)
434	On ignore les colonnes masquées qui n'existent plus ou qui sont obligatoires
436-440	Le tri mémorisé est repris s'il correspond à une colonne existante
442	`CalculerColonnes()` (ligne 576) : liste des colonnes visibles et largeur totale du tableau
446	Premier appel au **Service** : `ConsultationDevisService.LireContexteAsync` (Services, ligne 20) exige une session (`ExigerAsync`), puis demande aux **Données** si le profil est limité à ses propres devis (B000F.ILIMIT). Écran → Service → Données
447	Représentant imposé : la zone est pré-remplie (et grisée dans le HTML, ligne 66)
448-452	Premier passage : compte général et compte client par défaut repris de la session
455-458	En cas d'erreur (IBM i injoignable…) : notification rouge, l'écran s'affiche quand même
459	`Actualiser()` (ligne 606) : calcule les lignes à afficher (filtres + tri). Au premier passage `Etat.Lignes` est `null` : liste vide

> **Aucun devis n'est chargé à l'ouverture.** C'est une règle de la charte : l'utilisateur saisit ses critères puis lance la recherche.

`async` / `await` : `await` attend la fin d'une opération longue (lecture de fichier, requête DB2) **sans bloquer le serveur**, qui peut servir les autres utilisateurs pendant ce temps. Une méthode qui contient des `await` est marquée `async` et renvoie une `Task`.

8.4 Le rendu HTML (lignes 18 à 362) : la fenêtre s'affiche

Une fois `OnInitializedAsync` terminé, Blazor « exécute » la partie HTML de haut en bas pour produire la fenêtre, et l'envoie au navigateur via SignalR.

Lignes	Partie de la fenêtre	Ce qu'il faut remarquer
18	`<PageTitle>`	Titre de l'onglet du navigateur
20-34	Barre d'outils	« Détail du devis » : `disabled="@(Selection is null)"`, grisé tant qu'aucune ligne n'est sélectionnée. Les autres boutons sont grisés (« prochaine étape »)
36	`<form @onsubmit="RechercherAsync">`	Entrée dans les critères ou bouton Rechercher → `RechercherAsync` (chapitre 9)
37-88	Critères principaux	`@bind="C.Numero"` : **liaison dans les deux sens** entre la zone et la variable. Équivalent d'une zone liée à une rubrique en WinDev, sans `EcranVersFichier` à écrire
66-67	Zone Représentant	Grisée si un représentant est imposé (lu ligne 447)
89-97	Boutons	Rechercher (`type="submit"`, grisé pendant le chargement), Effacer (`@onclick="Effacer"`), Plus de critères (bascule `Etat.PlusDeCriteres`)
99-183	`@if (Etat.PlusDeCriteres)`	Les critères avancés (commercial, formats, papier, fabrication) n'existent dans la page que si on les a demandés
186-215	Compteur	Si `Etat.Lignes` est `null` : « Saisissez au moins un critère principal… ». Sinon : nombre de devis, et l'avertissement des 10 000 lignes maximum
216-237	Menu « ▦ Colonnes »	Une case par colonne (`@foreach`) pour masquer / réafficher
238-243	Légende	Offre, En commande, Échec
246-260	Filtres actifs	Affichés seulement s'il y a des filtres de colonnes
263	`<table @ref="tableGrille">`	`@ref` mémorise la balise dans `tableGrille`, pour la passer au JavaScript après l'affichage
264-269	`<colgroup>`	Largeur de chaque colonne visible (préférences ou largeur d'origine)
272-278	Titres de colonnes	Composant commun `<EnteteColonne>` (Web.Commun) : on lui passe des paramètres et des **procédures de rappel** (`OnTri="TrierAsync"`, `OnFiltre="Filtrer"`)
282	`<Virtualize Items="lignesAffichees">`	Ne produit **que les lignes visibles** à l'écran (plus une marge) : la grille reste fluide avec 10 000 devis
283-286	Une ligne `<tr>`	Couleur selon l'état (échec, commande, offre). Clic = sélection, double-clic = `Ouvrir`, clic droit = `OuvrirMenu`
287-311	Les cellules	Colonne « N° devis » avec son badge ; les autres via `Texte(c.Cle, l)` (ligne 639) et une bulle `Bulle(...)` (ligne 674)
314-321	Lignes vides	« Aucun devis ne correspond… » selon le cas
324-331	Zone d'attente	Au premier affichage (`Etat.Lignes` null) : rappel des critères principaux
336-362	Menu contextuel	N'existe que si `menu` n'est pas `null` (après un clic droit), positionné à l'endroit du clic

> **Différence majeure avec WinDev :** on ne modifie jamais un champ « à la main » (pas de `SAI_Numero = ...` ni de `..Etat = Grisé`). On modifie des **variables**, et le HTML les relit à chaque affichage : `disabled="@(Selection is null)"` se réévalue tout seul.

8.5 OnAfterRenderAsync (lignes 462 à 474) : après le premier affichage

```
protected override async Task OnAfterRenderAsync(bool premierAffichage)
{
    if (!premierAffichage) return;                                    // 464
    grille = new GrilleJs(JS) { LargeurChangee = ChangerLargeurAsync };  // 465
    try
    {
        await grille.ActiverAsync(tableGrille);                       // 468
    }
    catch (JSException ex) { Notifications.Erreur(...); }           // 470-473
}
```

Ligne	Ce qui se passe
464	Cette méthode est appelée après **chaque** affichage ; on n'agit qu'au premier
465	Création du pont avec `grille.js` (Web.Commun). `LargeurChangee = ChangerLargeurAsync` : quand l'utilisateur aura élargi une colonne, le JavaScript rappellera `ChangerLargeurAsync` (ligne 557)
468	Active le redimensionnement à la souris sur la balise `<table>`. Impossible avant : tant que la page n'est pas affichée, la balise n'existe pas dans le navigateur

À ce stade, **la fenêtre est affichée et prête**. L'écran attend les actions de l'utilisateur.

9. Après l'affichage : événements et recherche

9.1 Les événements de l'écran

Action de l'utilisateur	Déclenché par (HTML)	Procédure (ligne)
Entrée dans un critère ou bouton Rechercher	`@onsubmit`, ligne 36	`RechercherAsync` (481)
Bouton Effacer	`@onclick`, ligne 91	`Effacer` (507)
Clic sur une ligne	ligne 284	`Etat.Selection = l.Cle` (directement dans le HTML)
Double-clic sur une ligne, bouton Détail	lignes 285, 21	`Ouvrir` (516) → écran `FicheDevis`
Clic droit sur une ligne	ligne 286	`OuvrirMenu` (523)
Bouton de tri d'une colonne	`EnteteColonne`, ligne 275	`TrierAsync` (537)
Filtre d'une colonne	`EnteteColonne`, ligne 276	`Filtrer` (586)
Menu Colonnes	lignes 228, 234	`BasculerColonneAsync` (548), `RetablirAffichageAsync` (567)
Élargir une colonne à la souris	`grille.js`	`ChangerLargeurAsync` (557)

> Après chaque procédure déclenchée par un événement, Blazor **réaffiche automatiquement** l'écran et n'envoie au navigateur que ce qui a changé. Pas d'équivalent à appeler de type `AffichageChamp`. Exception : un rappel venu du JavaScript (ligne 562, `StateHasChanged`) doit demander lui-même le réaffichage.

9.2 La recherche : RechercherAsync (lignes 481 à 504)

```
private async Task RechercherAsync()
{
    chargement = true;                                          // 483
    try
    {
        var resultat = await Consultation.RechercherAsync(C);   // 486
        Etat.Lignes = resultat.Lignes.ToList();                 // 487
        Etat.Tronque = resultat.Tronque;                        // 488
        Etat.Selection = null;                                  // 489
        Actualiser();                                           // 490
    }
    catch (RegleMetierException ex) { await Dialogues.AlerterAsync(ex.Message); }   // 492-495
    catch (Exception ex) { Notifications.Erreur($"Recherche impossible : ..."); }  // 496-499
    finally { chargement = false; }                             // 500-503
}
```

Ligne	Ce qui se passe
483	Bouton Rechercher grisé et « Chargement… » affiché
486	Appel du **Service** avec les critères (voir 9.3)
487-489	Résultat rangé dans `Etat` (conservé pour le retour du détail), sélection effacée
490	Filtres de colonnes et tri appliqués en mémoire → `lignesAffichees` → la grille se remplit
492-495	Règle métier non respectée (ex. « Vous devez renseigner au moins un critère. ») → fenêtre d'information. Équivalent de `Info()`
496-499	Erreur technique → notification rouge
500-503	`finally` s'exécute **dans tous les cas** : le bouton est réactivé, même après une erreur

9.3 Côté Service puis Données

`src/Modules/NetPrint.Modules.Ventes/Services/ConsultationDevisService.cs`, méthode `RechercherAsync` (lignes 29 à 43) :

Ligne	Ce qui se passe
31	`utilisateur.ExigerAsync()` : la société vient **de la session**, jamais de l'écran (règle 4)
32	Copie des critères : l'écran n'est pas modifié par le service
35-36	Représentant imposé réappliqué **côté serveur** : impossible de le contourner depuis le navigateur
39	`Controler(c)` (ligne 46) : la procédure Controle du WinDev, mêmes messages ; lève une `RegleMetierException`
41	Appel des **Données** : `ConsultationDevisDb2.RechercherAsync` (Donnees, ligne 34) construit le SQL paramétré sur DB2
42	`VersLigneListe` (ligne 67) : transforme chaque ligne brute DB2 en ligne d'écran (formats, nom client ou prospect, valeur du devis…)

> On retrouve le découpage de l'étape 6 du chapitre 1 : **Écran** (`ConsultationDevis.razor`) → **Service** (règles) → **Données** (SQL). L'écran ne contient aucun SQL : le test d'architecture le vérifie.

9.4 Ouvrir un devis et revenir

`Ouvrir` (ligne 516) mémorise la sélection puis appelle `Navigation.NavigateTo("ventes/devis/12647/0/2/C")` : le routeur trouve `FicheDevis.razor` (`@page "/ventes/devis/{Numero:int}/..."`). Au retour, `ConsultationDevis.razor` est **recréé** : `OnInitializedAsync` repasse, mais comme `EtatConsultationDevis` est *Scoped*, critères, lignes, tri, filtres et sélection sont retrouvés tels quels.

10. Points d'arrêt et correspondance WinDev

Où mettre les points d'arrêt pour tout suivre

#	Fichier	Ligne	Moment
1	`MainLayout.razor`	96	Clic dans le menu utilisateur (`Aller`)
2	`MainLayout.razor`	78	Changement d'adresse (`Changement`)
3	`ConsultationDevis.razor`	431	Premier code de l'écran (`OnInitializedAsync`) — le plus utile
4	`ConsultationDevis.razor`	464	Après le premier affichage (`OnAfterRenderAsync`)
5	`ConsultationDevis.razor`	483	Clic sur Rechercher (`RechercherAsync`)
6	`ConsultationDevisService.cs`	31	Entrée dans le Service ; F11 sur la ligne 41 pour descendre dans les Données

Les points d'arrêt se posent directement dans le bloc `@code { }` d'un fichier `.razor`, comme dans un `.cs`.

Correspondance WinDev

Blazor	WinDev
`Navigation.NavigateTo(url)`	`Ouvre(FEN_...)`
`MainLayout.razor` et `@Body`	Fenêtre mère / modèle de fenêtre
`@page "/ventes/devis"`	Nom de la fenêtre
`@inject`	Variables globales, procédures d'un autre projet
Variables du bloc `@code`	Variables globales de la fenêtre
`OnInitializedAsync()`	Code « Fin d'initialisation » de la fenêtre
`OnAfterRenderAsync(true)`	Code exécuté une fois la fenêtre affichée
`@bind="C.Numero"`	Zone liée à une rubrique
`@onclick="Effacer"`	Code de clic du bouton
`RechercherAsync()`	Code de clic du bouton Chargement
`Dialogues.AlerterAsync(...)`	`Info(...)`
`disabled="@(...)"`	`..Etat = Grisé`
`EtatConsultationDevis` (Scoped)	Valeurs conservées entre deux ouvertures de la fenêtre
`DisposeAsync()`	Code de fermeture de la fenêtre

> **Exercice :** posez les points d'arrêt 3, 4 et 5, lancez, connectez-vous et cliquez sur la tuile « Consultation devis ». Observez l'ordre d'arrêt (3 puis 4), puis saisissez un n° de devis et faites Entrée (5). Avancez avec F10 jusqu'à la ligne 486 et faites F11 pour entrer dans le Service.